Monte Carlo Simulation - Portfolio loss and importance sampling

Thomas Sejer Canham

October 11, 2026

Aarhus University

Contents

1 Introduction **1**

2 Crude Monte Carlo Simulation **1**

3 Control Variable **2**

4 Importance-Sampling **5**

5 Comparison of the methods **7**

6 Appendix **8**

1 Introduction

In this project, we will investigate the use of Monte Carlo simulation (MCS) to estimate the exceedance probability $\mathbb{P}(L > x)$ of a loss

$$L = \delta^T \Delta S + \Delta S^T \Gamma \Delta S,$$

where, *delta* $\in \mathbb{R}^m$, $\Gamma \in \mathbb{R}^{m \times m}$ and $\Delta S \sim \mathcal{N}(0, \Sigma_S)$ is a (multivariate) normal random variable. The exceedance probability is the probability that the loss L exceeds a certain threshold $x = 10$.

In the parts (a), (b) and (c), we will focus on the one-dimensional case, $m = 1$. These three parts will compare crude MCS with MCS where we introduce control variates and importance samplingm respectively. In part (d), the differences of these methods are visuallized by comparison of the confidence intervals of the estimates that are obtained from each method.

Parameters for the various codes snippets can be seen in the appendix in Listing 4.

2 Crude Monte Carlo Simulation

In this section, we will implement a crude Monte Carlo simulation to estimate the exceedance probability $\mathbb{P}(L > x)$ for the one-dimensional case ($m = 1$). We are interested in $P(L > x)$. We may introduce the indicator function $\mathbb{I}(L > x)$ of the event $L > x$. If we compute the expectation of the indicator function, we obtain the exceedance probability

$$\mathbb{P}(L > x) = \mathbb{E}[\mathbb{I}(L > x)] = 0 \cdot \mathbb{P}(L > x) + 1 \cdot \mathbb{P}(L > x) = \mathbb{P}(L > x), \tag{1}$$

by the bernoulli property of the indicator function. We estimate the exceedance probability by use of the following Monte Carlo estimator

$$\hat{\ell} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(L_i > x) \xrightarrow[N \to \infty]{a.s} \mathbb{E}[\mathbb{I}(L > x)], \tag{2}$$

by the law of large numbers. The crude Monte Carlo simulation is implemented in the code snippet seen in Listing 1. A estimate of

$$\hat{\ell} \approx 0.0103$$

is found with a variance of $s^2 = 1.03 \cdot 10^{-7}$.

Listing 1: Python code for crude MCS

```
"""
Code for task (a) - Crude monte carlo

"""

from parameters import *

def indicator(x, L):
    # Exceedance of x
    return L > x

def L(Delta_s, delta, gamma):
    """L for N simulations

    input:
    N (int): Number of simulations
    Delta_S: N(0, Sigma_S) of shape (m, N) samples
    gamma: vector of size m
    delta: vector of size m
    """
    # Returns (N,) losses
    L = delta.T @ Delta_s + np.sum(Delta_s * (gamma @ Delta_s), axis=0)
    return L

```

```python
def crude_monte_carlo(N, Delta_s= None):
    """
    Crude Monte Carlo simulation to estimate the probability of exceedance of x.

    Parameters:
    N (int): Number of simulations

    Returns:
    float: Estimated probability of exceedance
    """

    if Delta_s is None:
        Delta_s = np.random.multivariate_normal(np.zeros(m), sigma_s, size = N).
            T

    # Simulate N outcomes of L
    loss = L(Delta_s, delta, gamma)

    # Mean of indicator converges a.s to P(L > x)
    exeedance = indicator(x, loss)
    # Estimate
    exeedance_prop = np.mean(exeedance)
    # Variance of estimate
    variance = np.var(exeedance) / N
    return exeedance_prop, variance

if __name__ == "__main__":
    print("Crude Monte Carlo simulation: \n")
    np.random.seed(0)
    N = 100000
    # Estimate and variance of estimator
    l, s2 = crude_monte_carlo(N)
    print("Estimated probability of exceedance of x: ", l)
    print("Variance of MC estimator: ", s2)
```

3 Control Variable

A control variable is now introduced with the aim of reducing the variance of the Monte Carlo estimator. The estimator is now given by

$$\ell_{CV} = Y - \beta(Z - \mathbb{E}[Z]), \tag{1}$$

where $Y = \mathbb{I}(L > x)$ is the original estimator, Z is the control variable and β is a constant. It follows from proposition 5.1, that the optimal value of β is given by

$$\beta^* = \frac{\mathrm{Cov}(Y, Z)}{\mathrm{Var}(Z)}.$$

When considering the choice of a control variable, we should aim to choose a variable that satisfies the following properties:

1. Closed form expression for the expectation of the control variable $\mathbb{E}[Z]$.
2. The control variable should be as correlated with the original estimator Y as possible.
3. The reduction in variance should be significant enough to justify the additional computational cost of calculating the control variable.

Given L, two choices of control variables are natural. The first one is the linear term of L given by:

$$Z_1 = \delta^T \Delta S,$$

and the second one is the quadratic term of L given by:

$$Z_2 = \Delta S^T \Gamma \Delta S.$$

Since $m, \Gamma = 1$, and the fact that $Delta_S$ is standard normal, $Z_1 \sim \delta\mathcal{N}(0,1)$ and $Z_2 \sim \chi^2(1)$, and as such their expectations are given by

$$\mathbb{E}[Z_1] = 0, \quad \mathbb{E}[Z_2] = 1.$$

To test which control variable is the best, a small pilot study is conducted to find the sample correlation between the original estimator Y and the two control variables Z_1 and Z_2, yeilding

$$\text{Corr}(Y, Z_1) \approx 0.25, \quad \text{Corr}(Y, Z_2) \approx 0.37.$$

Given the results of the pilot study, and the fact that Z_2 is not much more computationally expensive to compute than Z_1, Z_2 is chosen as a control variable. This leads to a reduction in variance of approximately 26% compared to the crude MCS. Since crude MCS has a $\sqrt{N}$ convergence rate, and we we roughly double the number of operations by computing the control variable, it seem reasonable to stick with crude MCS and double the number of samples, in order to achieve a reduction in variance of the order $\sqrt{2}$.

Listing 2: Python code for control variable MCS

```
"""
We introduce a control variable to reduce the varinace.
"""
# Same as in (a)
from a import L, indicator, crude_monte_carlo
from parameters import *

# Use Q as control variable
def Q(Delta_s,  delta):
    return delta.T @ Delta_s

def compute_optimal_coefficients(Y, Z):
    """
    Y: target, shape (1, N)
    Z: control variates, shape (k, N)
       where k is the number of control variates.

    Returns:
        beta_opt: shape (k,)
    """
    Y = np.asarray(Y).reshape(-1)
    Z = np.atleast_2d(Z)

    if Z.shape[1] != Y.size:
        raise ValueError("Y and Z must have the same number of simulations.")

    cov = np.cov(np.vstack([Y, Z]), ddof=1)

    cov_YZ = cov[0, 1:]
    cov_ZZ = cov[1:, 1:]

    beta_opt = np.linalg.solve(cov_ZZ, cov_YZ)
    return beta_opt

def compute_correlation(Y, Z):
    """
    Compute the sample correlation coefficient for k different control variates.
    Y: target, shape (1, N)
    Z: control variates, shape (k, N)
       where k is the number of control variates.

    Returns:
        correlation: shape (k,)
    """
    Y = np.asarray(Y).reshape(-1)
    Z = np.atleast_2d(Z)

```

```
    if Z.shape[1] != Y.size:
        raise ValueError("Y and Z must have the same number of simulations.")

    cov = np.cov(np.vstack([Y, Z]), ddof=1)

    cov_YZ = cov[0, 1:]
    var_Y = cov[0, 0]
    var_Z = np.diag(cov[1:, 1:])

    correlation = cov_YZ / np.sqrt(var_Y * var_Z)
    return correlation




def control_monte_carlo(N, Delta_s = None):
    """
    Control variate Monte Carlo simulation to estimate the probability of
        exceedance of x.

    Parameters:
    N (int): Number of simulations

    Returns:
    float: Estimated probability of exceedance
    """

    if Delta_s is None:
        Delta_s = np.random.multivariate_normal(np.zeros(m), sigma_s, size = N).
            T
    # Simulate N outcomes of L
    loss = L(Delta_s, delta, gamma)

    # Compute control variable

    #Linear term, corr aprrox 0.25
    # #Z = Q(Delta_s, delta)

    # Quadratic term, corr approx 0.4
    Z = L(Delta_s, delta, gamma) - Q(Delta_s, delta)
    # Expectation of control variable
    #E_Z = 0 # Linear term is N(0,1) * delta -> E[Z] = 0
    E_Z = 1 # Quadratic term i Chi_squared(1) -> E[Z] = 1


    # Mean of indicator converges a.s to P(L > x)
    exeedence = indicator(x, loss)


    # optimal coeffcient
    beta_opt = compute_optimal_coefficients(exeedence, Z)

    # Control estimator
    exeedence_tilde = exeedence -  beta_opt*(Z - E_Z)

    # Estimate
    exeedance_prop = np.mean(exeedence_tilde)

    # Variance of estimate
    variance = np.var(exeedence_tilde) / N

    return exeedance_prop, variance

if __name__ == '__main__':
```

```
    # Correlation subrun
    np.random.seed(0)
    M = 1000
    Delta_s = np.random.multivariate_normal(np.zeros(m), sigma_s, size=M).T
    loss = L(Delta_s, delta, gamma)
    Y = indicator(x, loss)
    # Compute correlation between linear approximation Q, and second term of L
    Z = np.zeros((2,M))
    Z[0,:] = Q(Delta_s, delta)
    Z[1,:] = loss - Q(Delta_s, delta)
    correlation = compute_correlation(Y, Z)
    print("Sample Correlation of linear term and exceedance indicator: ",
        correlation[0])
    print("Sample Correlation of quadratic term and exceedance indicator: ",
        correlation[1])

    print("Control variate Monte Carlo simulation: \n")
    N = 10000
    Delta_s = np.random.multivariate_normal(np.zeros(m), sigma_s, size=N).T
    # Estimate and variance of estimator
    l_crude, s2_crude = crude_monte_carlo(N, Delta_s)
    l_control, s2_control = control_monte_carlo(N, Delta_s)
    print("Estimated probability of exceedance of x using control variable: ",
        l_control)
    print("Variance of MC estimator using control: ", s2_control)
    print()
    print("Variance reduction factor: ", s2_crude / s2_control)
```

4 Importance-Sampling

Since we are interested in estimating a relatively rare event, we can use importance sampling to reduce the variance. We are interested in designing a new distribution under which the event $L > x$ is likely to occur. For numerical stability, it is desirable to choose a distribution that is not too far from the original distribution. If we inspect L, it is compromised of a linear and a quadratic term. We may consider if we want to shift the mean of Δ_S upwards or downwards. Since we have a quadratic term, that is always positive, shifting the mean in a negative direction, will be somewhat dampened by the linear term, as such we would require larger shifting in the negative direction to observe the same effect, therefore, a positive shift is chosen. The expexctation of the loss L is given by

$$\mathbb{E}[L] = \mathbb{E}[\delta^T \Delta S + \Delta S^T \Gamma \Delta S] = \mathbb{E}[\delta^T \Delta S] + \mathbb{E}[\Delta S^T \Gamma \Delta S] = 0 + 1 = 1,$$

Now we would like to shift the mean of ΔS such that the expectation of L is equal to the threshold $x = 10$, such that the rare event $L > x$, is no longer rare. We do this by setting the expectation of L under the new distribution $\mathbb{Q}$ to be equal to $x = 10$:

$$\mathbb{E}_{\mathbb{Q}}[L] = \mathbb{E}[\delta^T(\Delta S + \mu) + (\Delta S + \mu)^T \Gamma(\Delta S + \mu)] = 10.$$

as adding a mean μ to ΔS, will shift its mean by μ, so for $m = 1$

$$\mathbb{E}_{\mathbb{Q}}[L] = 2\mu + \mathbb{E}(\Delta S^2) + \mu^2 = 2\mu + 1 + \mu^2 = 10 \implies \mu(\mu + 2) = 9.$$

The positive solution to this equation is $\sqrt{(10)} - 1 \approx 2.16$, and in accordance with above reasoning, we see that a negative shift would require $\mu \approx -4.16$, which is considerably larger in magnitude.

Using this design choice, variance reduction on the order of 3000% is achieved. As such, importance-sampling offers a significant improvement compared to crude MCS, far outweighing the additional computational cost of computing the likelihood ratios. The code snippet for the importance sampling MCS can be seen in Listing 3.

Listing 3: Python code for importance sampling MCS

```python
"""
Monte Carlo with importance sampling for variance reduction
"""
from a import indicator, L
from parameters import *

def likilihood_ratio(X, mu_p, sigma_p, mu_q, sigma_q):
    """
    We use another normal distribution with greater mean as importance sampling.
    Compute dP(X) / dQ(X), where:
    Under P: X_i ~ N(m_P, variance)
    Under Q: X_i ~ N(m_P + drift, variance)
    """
    # Compute the likelihood ratio
    p = (1 / np.sqrt(2 * np.pi * sigma_p**2)) * np.exp(-0.5 * ((X - mu_p) / sigma_p)**2)
    q = (1 / np.sqrt(2 * np.pi * sigma_q**2)) * np.exp(-0.5 * ((X - mu_q) / sigma_q)**2)
    return p / q


def importance_monte_carlo(N, sample = None):
    """
    Importance Monte Carlo simulation to estimate the probability of exceedance of x.

    Key features are that we sample from another distribution that makes the probablily of
    exceedance more likely, and then we normalize the samples by the likelihood ratio.

    Parameters:
    N (int): Number of simulations

    Returns:
    float: Estimated probability of exceedance
    """
    # Draw samples - this time from the importance distribution (expandable for m > 1)
    if sample is None:
        importance_sample = np.random.multivariate_normal(mu_q, sigma_s, size = N).T

    # Compute likihood ratio for each sample


    weights = likilihood_ratio(importance_sample, mu_p, sigma_p, mu_q, sigma_q)

    # Simulate N outcomes of L
    loss = L(importance_sample, delta, gamma)

    # Mean of indicator converges a.s to P(L > x), weighted by the likihood ratio
    exeedance = indicator(x, loss) * weights
    # Estimate
    exeedance_prop = np.mean(exeedance)
    # Variance of estimate
    variance = np.var(exeedance) / N
    return exeedance_prop, variance

if __name__ == '__main__':
    N = 10000
    np.random.seed(0)
    from a import crude_monte_carlo
```

```
    l_crude, s2_crude = crude_monte_carlo(N)
    print("Importance sampling Monte Carlo simulation: \n")
    l_importance, s2_importance = importance_monte_carlo(N)
    print("Estimated probability of exceedance of x using importance sampling: "
        , l_importance)
    print("Variance of MC estimator using importance sampling: ", s2_importance)
    # Variance reduction factor
    print("The variance reduction factor of importance sampling: ", s2_crude /
        s2_importance)
```

5 Comparison of the methods

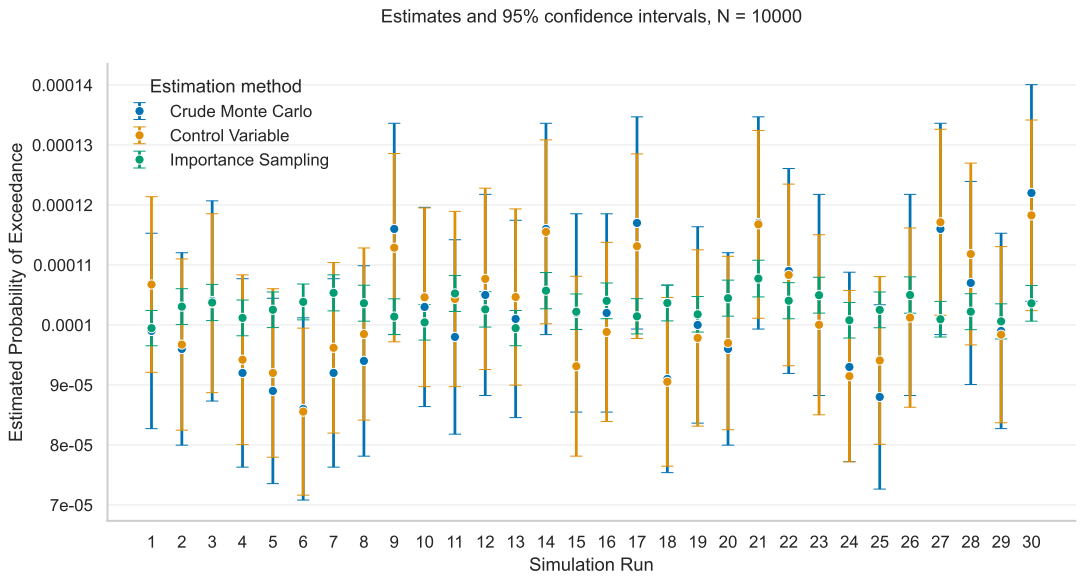


Figure 1: Comparison of confidence intervals for crude MCS, control variable MCS and importance sampling MCS. The confidence intervals are based on 30 simulations with $N = 10^4$ samples each.

6 Appendix

Listing 4: parameters.py - Parameters for the various code snippets

```python
import numpy as np
### (a), (b) ###

m = 1
# delta, gamma is a column vector of size m
delta = np.ones((m, 1)) * 2
gamma = np.ones((m,m))
x = 10
sigma_s = np.ones((m, m))

### (c) ###

# Q measure paramters
mu_q = np.ones(m) * np.sqrt(10) - 1
sigma_q = sigma_s

# P measure parameters
mu_p = np.zeros(m)
sigma_p = sigma_s

######## (d) ########
import seaborn as sns
from scipy.stats import norm

Nd = 10000

SIMULATION_RUNS = 30
plot_n = np.arange(1, SIMULATION_RUNS + 1)

methods = ["Crude Monte Carlo", "Control Variable", "Importance Sampling"]
colors = sns.color_palette("colorblind", 3)

def confidence_interval(s, l , N, level = 0.05):
    """Compute confidence interval, equation (1.2)"""
    z_alpha = norm.ppf(1 - level)
    return (l - z_alpha*s / np.sqrt(N), l + z_alpha*s / np.sqrt(N) )
```